When Model Identity Is Dropped: A Controlled Join Audit of a Multi-Agent Contamination Artifact

Ezekiel Ologunde

Abstract

Multi-model agent studies reuse task identifiers across backends. Aggregation must retain the identity of the execution whose outcome is being measured. We audit one pinned public contamination-analysis artifact. Independent table checks reproduce 614 trace records across 32 tasks and the released 15.3% silent-corruption label share; 624 token records include ten keys without trace matches. The optional correctness-analysis path drops model identity and joins by task and perturbation alone. A controlled two-model counterexample expands two observations into four: high-cost correctness and low-cost incorrectness both become 50%, although each intended stratum has one observation and a rate of 100%. An identity-preserving one-to-one join recovers those fixture values, rejects ambiguous keys, and retains missing outcomes as unknown. Six checks pass. The acquired artifact does not include the raw correctness records needed to estimate the effect on published correctness results. This is a revision-specific reproducibility finding and ordinary join-contract repair, not a new memory defense, a new algorithm or an independent validation of the original model outcomes.

Keywords

agent evaluation, reproducibility, data joins, execution identity, contamination

1 Question and Prior Work

Mazhar, Suri and Galhotra study how artifact perturbations propagate through fixed multi-agent workflows [1]. Their paper distinguishes trace divergence, silent semantic corruption, recovery and operational cost. It reports 614 paired runs across 32 GAIA tasks and three model backends, and supplies public analysis tables and scripts. These are already established contamination measurements; our question concerns their aggregation boundary.

When several models execute the same task and perturbation, task identity alone does not identify an execution. We ask whether the released tables can reproduce basic counts, what coverage a token/trace join retains, and whether optional correctness aggregation preserves model identity. Our candidate contribution is an executable counterexample against one pinned script and a strict input contract. The join technique itself is standard. Standalone novelty and venue suitability remain unestablished.

2 Acquisition and Audit Design

We pin `anna-mazhar/trace-level-contamination-mas` at commit `a6aaf30c`..., retaining source inventories, retrieval URLs, sizes and SHA-256 receipts. We acquire nine CSVs, the author statistics script and retained reference outputs; all 12 selected hashes verify. The tables contain per-model trace metrics, token usage and perturbation aggregates. They are inert inputs. No workflow, model endpoint, attack or tool invocation is executed.

Table 1: Measured released-table coverage. "Matched" counts unique model/task/perturbation keys.

Source label	Trace rows	Token rows	Matched
GPT	179	188	179
LLaMA	218	218	218
Qwen	217	218	217
Total	614	624	614

Our independent table analysis uses the Python standard library. The original aggregation script is additionally run with pandas 3.0.1 and NumPy 2.3.5. Its optional raw-correctness directory is absent in the acquired release; we point that argument to a nonexistent local path so correctness-dependent fields remain unavailable. We inspect schemas, table sizes and source joins before writing the audit record. We record a separate controlled-test protocol before running its counterexample. This is local exploratory auditing, not external preregistration or blinded discovery.

3 Available-Table Reanalysis

Table 1 reports observed coverage. Within each model, `task_id` and `perturbation_type` form unique keys in both tables. All trace records match tokens. Nine GPT and one Qwen token records have no trace match; their presence does not, by itself, establish an erroneous experiment or a missing adverse result.

The 614 trace rows cover 32 task identifiers. The released manifestation labels contain 94 silent-corruption cases ($94/614 = 15.309\%$) and 247 strategy-reroute/extended-execution cases (40.228%). The author script rounds the latter component percentages before adding them and reports 40.3%; direct pooling rounds to 40.2%. This small rounding distinction is not evidence against the qualitative result. Our independent counts agree with the underlying released category totals.

The acquired tables do not provide per-run independent correctness or reference answers. Answer change is not equivalent to incorrectness, and a manifestation label is not a substitute for raw adjudication. We therefore do not construct a correctness detector or infer correctness-sensitive rates from these fields. The retained reference log describes loading additional correctness records, but those records were not in the inspected public tree. Reproducing available aggregates is consequently a narrower result than validating the study end to end.

4 Controlled Correctness-Join Counterexample

The source loader reads optional `summary.json` records and emits task, perturbation and correctness fields without retaining model identity. Its cost/correctness merge uses only task and perturbation, even though token rows retain source/model fields. Thus two

correctness records for the same task/perturbation can match each token row.

Our controlled fixture has two named models and one shared task/perturbation. Model A has a token multiplier of 3.0 and an explicit correct label; model B has a multiplier of 1.0 and an explicit incorrect label. Under the source thresholds, A is high cost (> 2) and B low cost (< 1.2). Each intended stratum contains one observation. The expected high-cost correctness and low-cost incorrectness are both 100%.

We invoke the pinned author's aggregation function, retaining its other required trace fields in the fixture. Its correctness merge produces two matched rows per stratum and reports 50% for both rates. This is a deterministic cross-model label mix: each token observation matches both correctness labels. A strict join on model, task and perturbation instead yields one row per stratum and the intended 100% rates. These values describe the fixture, not a measured change in natural agent reliability.

An equal-correctness negative control makes both models correct. The source and strict join then agree on rates (100% high-cost correctness, 0% low-cost incorrectness), while the source still doubles the stratum counts. A correct aggregate rate alone therefore need not establish a valid observation count.

5 Repair Contract and Verification

Our small repair requires nonempty model/task/perturbation identifiers on both sides, explicit Boolean correctness on labeled records, and unique complete keys. It rejects ambiguous duplicate keys rather than silently multiplying observations. A token row without a correctness match remains `unknown`; it is excluded from labeled rate denominators and counted separately. For repeated trials, a run identifier must extend this key. This artifact-specific contract is not a universal trial-identity schema.

Six checks pass: the counterexample, reversed input order, equal-label control, missing identity rejection, unmatched outcome preservation, and duplicate-key rejection. Source-byte verification ties the invoked function to the recorded revision. These checks support the join behavior and its repair, not the scientific validity of the original outcome labels.

6 Limits and Reproducibility

The counterexample establishes a failure condition in the optional source-code path. We cannot determine its realized incidence or effect size in the original correctness analysis without the raw summary records and exact invocation. We do not claim that the paper's correctness percentages are wrong, that the reproduced count/manifestation results are invalid, or that a memory-defense gap has been solved. Model behavior and causal contamination were not independently tested.

The authored package preserves acquisition scripts, protocols, independent table results, the controlled fixture and strict join. Upstream code and CSVs remain separately reacquired material because redistribution terms were not independently established. The Overleaf package retains the ACM class and its license notices. No author contact, independent human adjudication, public submission or peer review is claimed. Generative AI assisted literature inspection, implementation and drafting.

The practical implication is narrow: model identity must survive from raw outcomes to joined cost statistics, and missing correctness must remain visible. The resulting technical note is a reproducibility audit of a particular artifact, with broader evaluation left to additional independently inspected studies.

References

[1] Anna Mazhar, Huzaifa Suri, and Sainyam Galhotra. 2026. Trace-Level Analysis of Information Contamination in Multi-Agent Systems. In *ACM Conference on AI and Agentic Systems (CAIS '26)*. https://doi.org/10.1145/3786335.3813147. Author artifact: https://github.com/anna-mazhar/trace-level-contamination-mas.